CPU PROGRAMMER USER MANUAL

32-bit Five-Stage Pipelined CPU | Verilog, Python Assembler, Vivado XSim and Nexys A7

Purpose. This guide explains how to configure the project paths, assemble a program, simulate it, and optionally program the Nexys A7 FPGA. Follow the path setup before starting simulation.

Important: the locations shown below are the current/reference locations used by the project author. They are specific to that computer, not universal defaults. If you install Vivado elsewhere or place the project in another folder or drive, update the paths in all the named files to your actual locations.

Current/reference locations

Item	Current configured location
Project root folder	D:\cpu_32bit
Vivado bin folder	D:\2025.1\Vivado\bin
Vivado project file	D:\cpu_32bit\cpu_32bit.xpr
Simulation batch file	D:\cpu_32bit\run_simulation_fresh.bat
Simulation Tcl file	D:\cpu_32bit\run_simulation_fresh.tcl
Generated HEX file	D:\cpu_32bit\program.hex
Simulation JSON result	D:\cpu_32bit\simulation_result.json

Before simulation, verify that these two files really exist on your computer:

```
D:\2025.1\Vivado\bin\vivado.bat
D:\cpu_32bit\cpu_32bit.xpr
```

If either file is in a different location, use that actual location in the settings described on the next pages.

Project overview

Property	Description
CPU architecture	32-bit datapath, five pipeline stages: IF, ID, EX, MEM and WB
RTL top for FPGA	nexys_a7_top
Simulation testbench	cpu_sim_tb
FPGA target	Digilent Nexys A7-100T, Artix-7 xc7a100tcsg324-1
Clock / UART	100 MHz / UART 921600 baud, 8-N-1
Software	Python 3 with Tkinter; pyserial for physical UART use

1. Configure paths for your computer

The previous “Vivado project not found” error is a path-configuration error: one of the scripts is looking in a folder where the project is not located. Fixing only one file is not enough. The BAT file, Tcl file, testbench macros and GUI-generated files must all point to the same project root.

1.1 Check your project root

On the reference setup, the project root is **D:\cpu_32bit**, and the project file is **D:\cpu_32bit\cpu_32bit.xpr**. If you extracted/cloned it elsewhere, substitute your actual project root everywhere below. For example, if your project root is **C:\Projects\cpu_32bit**, then the .xpr path becomes **C:\Projects\cpu_32bit\cpu_32bit.xpr**.

1.2 Find your Vivado installation

The configured Vivado directory is **D:\2025.1\Vivado\bin**. It is correct only if the file **D:\2025.1\Vivado\bin\vivado.bat** exists. Open File Explorer and verify it. If your copy of Vivado is installed in another directory, find the **bin** folder that directly contains **vivado.bat**, and use that directory in the VIVADO setting.

For example only: if your actual file is **C:\Xilinx\Vivado\2025.1\bin\vivado.bat**, set VIVADO to **C:\Xilinx\Vivado\2025.1\bin**. Do not copy that example unless it matches your installation.

1.3 Update run_simulation_fresh.bat

Open **run_simulation_fresh.bat** in Notepad or VS Code. These are the current/reference settings:

```
set "VIVADO=D:\2025.1\Vivado\bin"
set "PROJECT=D:\cpu_32bit\cpu_32bit.xpr"
set "TCL=D:\cpu_32bit\run_simulation_fresh.tcl"
set "RESULT=D:\cpu_32bit\simulation_result.json"
cd /d D:\cpu_32bit
```

If your own paths differ, replace every value with the corresponding location on your computer. Also update the BAT file’s program.hex existence check so it tests for **your-project-root\program.hex**. VIVADO is the directory containing vivado.bat; PROJECT is the full path to the .xpr file; TCL is the full path to the Tcl script; RESULT is where the JSON result is expected; and cd /d selects the project working directory.

Example if your project is in C:\Projects\cpu_32bit

```
set "VIVADO=C:\Xilinx\Vivado\2025.1\bin"
set "PROJECT=C:\Projects\cpu_32bit\cpu_32bit.xpr"
set "TCL=C:\Projects\cpu_32bit\run_simulation_fresh.tcl"
set "RESULT=C:\Projects\cpu_32bit\simulation_result.json"
cd /d C:\Projects\cpu_32bit
```

This is an example, not an assumed location. Use the actual locations on your own PC.

2. Update the Tcl script and testbench

2.1 Update run_simulation_fresh.tcl

Open **run_simulation_fresh.tcl**. Change the PROJECT path, the program.hex check and the line that deletes an old result so they all use your own project root. Tcl paths can use forward slashes. Current/reference settings:

```
set PROJECT "D:/cpu_32bit/cpu_32bit.xpr"

if {![file exists $PROJECT]} {
    error "Vivado project not found: $PROJECT"
}
if {![file exists "D:/cpu_32bit/program.hex"]} {
    error "program.hex not found: D:/cpu_32bit/program.hex"
}

catch {file delete -force "D:/cpu_32bit/simulation_result.json"}
```

If your project root is different, replace every **D:/cpu_32bit** path in this script with your actual project root. Keep the path to the .xpr, input HEX, and JSON output consistent with the BAT file and testbench.

2.2 Update cpu_sim_tb.v (important)

Find the testbench at **cpu_32bit.srcs\sim_1\new\cpu_sim_tb.v** inside the project. Locate the CPU_PROGRAM_HEX_PATH and CPU_RESULT_JSON_PATH definitions. Set them to the actual location of program.hex and simulation_result.json. Example for the reference setup:

```
`ifndef CPU_PROGRAM_HEX_PATH
`define CPU_PROGRAM_HEX_PATH "D:/cpu_32bit/program.hex"
`endif

`ifndef CPU_RESULT_JSON_PATH
`define CPU_RESULT_JSON_PATH "D:/cpu_32bit/simulation_result.json"
`endif
```

For a project root of **C:\Projects\cpu_32bit**, use **C:/Projects/cpu_32bit/program.hex** and **C:/Projects/cpu_32bit/simulation_result.json** instead. If the macros already exist, edit those definitions rather than adding duplicates. Save the testbench and allow Vivado to recompile it.

A BAT/Tcl path correction alone may still fail if cpu_sim_tb.v keeps searching the old folder, such as D:/assembler. Check for and replace stale path references in the three simulation files.

2.3 Keep assembly and output files together

The Python GUI writes **program.hex** into the same directory as the currently saved .asm source. For the reference configuration, save a program such as **D:\cpu_32bit\instruction_test.asm**. The GUI then creates **D:\cpu_32bit\program.hex**. The testbench writes **D:\cpu_32bit\simulation_result.json**.

In the GUI's Simulation Settings, use **run_simulation_fresh.bat** as the simulation command and **simulation_result.json** as the result filename for the project-root workflow. If you place your assembly file elsewhere, either save it in the project root or update the workflow so the generated HEX is where the testbench expects it.

Path checklist

- The BAT file uses your real Vivado bin folder and actual .xpr path.
- The BAT and Tcl scripts point to the same project root, program.hex and JSON result.
- The testbench macros point to that same program.hex and JSON result.
- The GUI saves program.hex in the folder expected by the simulation scripts.

3. Start the assembler and run a simulation

3.1 Requirements

- Windows, Python 3 and Tkinter.
- AMD/Xilinx Vivado 2025.1 for XSim simulation and FPGA build/programming.
- pyserial is required for the physical FPGA UART connection: python -m pip install pyserial

3.2 Launch the GUI

Open Command Prompt or PowerShell in the project root. Keep assembler.py in the same folder as the GUI file. The UART-diagnostic GUI is recommended when inspecting physical serial traffic:

```
python cpu_assembler_gui_uart_diagnostic.py
```

The standard GUI can also be launched with:

```
python cpu_assembler_gui_fresh.py
```

3.3 Run a behavioral simulation

1. Save your assembly source (.asm) in the configured project root. On the reference setup, use D:\cpu_32bit.
2. Open the GUI and select **Simulator** mode.
3. Open Simulation Settings. Set the command to **run_simulation_fresh.bat** and result filename to **simulation_result.json** for the reference layout.
4. Click **Assemble** first to check syntax, or click **Build / Sim** (F7) to assemble and run the simulation.
5. The GUI writes program.hex. The BAT file launches Vivado, the Tcl file opens the project and starts cpu_sim_tb, and the testbench drives the program through the simulated UART loader.
6. When the run finishes, the GUI reads simulation_result.json and displays the program counter, halt status and R0-R31. If it fails, read the first meaningful error in Build Output rather than relying only on the final exit code.

3.4 Supported instructions

Category	Instruction forms
Register arithmetic	ADD Rd, Rs, Rt; SUB Rd, Rs, Rt; MUL Rd, Rs, Rt
Logic and compare	AND Rd, Rs, Rt; OR Rd, Rs, Rt; SLT Rd, Rs, Rt; SLTI Rt, Rs, immediate
Move and immediate	MOV Rd, Rs; MOVI Rt, immediate; ADDI Rt, Rs, immediate; SUBI Rt, Rs, immediate
Memory	LW Rt, offset(Rs); SW Rt, offset(Rs)
Control flow	BEQZ Rs, label; BNEQZ Rs, label; J label; HLT

Registers are R0-R31. R0 reads as zero and writes are ignored. MOVI zero-extends the encoded 16-bit value; ADDI, SUBI and SLTI use signed immediates. Use labels in LABEL: form and # for comments. For LW/SW, use word-aligned byte offsets such as 0, 4 and 8.

Example assembly program

```
MOVI R1, 5
MOVI R2, 1
LOOP:
SUBI R1, R1, 1
BNEQZ R1, LOOP
HLT
```

A final HALT state means the CPU reached HLT. R31 = 0 means PASS only if the assembly program uses that convention. The testbench can also report statuses such as TIMEOUT, FILE_ERROR, LOAD_TIMEOUT, MALFORMED_HEX or PROGRAM_TOO_LARGE.

4. Build and program the Nexys A7

4.1 Build the FPGA bitstream

1. Open the actual cpu_32bit.xpr file in Vivado 2025.1. For the reference setup, it is D:\cpu_32bit\cpu_32bit.xpr.
2. Confirm that nexys_a7_top is the synthesis top and that nexys_a7_top.xdc is enabled.
3. Run **Run Synthesis**, then **Run Implementation**, then **Generate Bitstream**.
4. Open Hardware Manager, connect to the Nexys A7, and program the generated bitstream.

Signal	Nexys A7 pin	Purpose
clk	E3	100 MHz clock
uart_rx_pin	C4	Computer-to-FPGA UART receive
uart_tx_pin	D4	FPGA-to-computer debug transmit
resetn	C12	Active-low reset input

4.2 Send an assembly program over UART

1. Connect the board by USB and program the bitstream. Reset the board before loading a new program so the loader returns to its waiting state.
2. Launch the GUI and select **Nexys A7** mode.
3. Click Refresh COM Ports, select the board COM port, set baud to **921600**, then click Connect.
4. Open or type the assembly program. Include HLT at the end and click **Send to FPGA** (F6 where enabled).
5. Monitor PC, HALT and register values. A “send complete” message confirms that bytes were written by the PC serial interface; incoming debug packets and the HALT state are stronger evidence that the CPU executed correctly.

4.3 UART debug packet

Byte range	Content
0	Header: 0xAA
1-4	32-bit program counter, most-significant byte first
5-132	R0-R31, four bytes per register, most-significant byte first
133	Halt flag: 0 = running, 1 = halted

Each debug packet is 134 bytes. Serial settings are 8 data bits, no parity, one stop bit (8-N-1). Do not open the same COM port in multiple applications simultaneously.

Useful behavior notes

- The PC uses byte addresses and advances by 4 for sequential instructions.
- Data-memory byte addresses should be word aligned for LW/SW; store to a location before loading it if a known value is required.
- The program loader uses 0xFFFFFFFF as its end marker; this is separate from the source-level HLT instruction.

5. Troubleshooting

Error or symptom	What to check
Vivado project not found: ...cpu_32bit.xpr	Confirm the .xpr exists at the exact path. Update PROJECT in both BAT and Tcl files.
Vivado not found: ...\vivado.bat	VIVADO must be the directory containing vivado.bat. Current reference: D:\2025.1\Vivado\bin. If yours differs, set your own path.
program.hex not found	Save the .asm in the configured project root and assemble again. Check the BAT/Tcl paths and CPU_PROGRAM_HEX_PATH in cpu_sim_tb.v.
XSim still searches D:/assembler	Search the BAT, Tcl and cpu_sim_tb.v files for D:/assembler and update all remaining paths to the current project root.
Simulation JSON missing	Check CPU_RESULT_JSON_PATH in cpu_sim_tb.v, the Tcl JSON cleanup/output path and BAT RESULT. All must use the same destination.
Simulation failed with exit code 1	The exit code alone is not the cause. Read the preceding XSim message in Build Output or inspect vivado.log.
GUI will not start	Confirm Python 3 and Tkinter are installed; run the command from the project folder and keep assembler.py beside the GUI.
COM port missing / no debug packets	Check USB, board power, driver, selected COM port, 921600 baud, XDC pins and reset state. Close other serial applications.
Simulation timeout	Check branch/jump loops and ensure the program can reach HLT. Inspect register values and testbench status.

Final path checklist before simulation

- Project file exists at your configured path (reference: D:\cpu_32bit\cpu_32bit.xpr).
- Vivado executable exists at the configured VIVADO location (reference: D:\2025.1\Vivado\bin\vivado.bat).
- BAT, Tcl and testbench all use the same project root. No old D:/assembler references remain if the project was moved.
- Your saved .asm file is in the folder that causes the GUI to create program.hex where the testbench expects it.
- GUI result filename, testbench output macro, Tcl cleanup path and BAT RESULT agree.

Remember: the paths D:\cpu_32bit and D:\2025.1\Vivado\bin are only the current/reference setup. A reader who clones the repository or installs Vivado elsewhere must replace each path with the actual folder locations on their own computer. Do not change only one of the files.

Publishing on GitHub

Keep README.md and CPU_Programmer_User_Manual.pdf in the repository root. The README links to this PDF. Include the RTL source, constraints, testbench, assembler, GUI and simulation scripts. A reader should update their local path settings before running the simulation.